

TP3 – Pruebas Funcionales y de Aceptación con Selenium



Materia: Prueba de Software

Alumnos:

- Roberto Andres Ruiz Pereira - DNI 37762826
- Luciano Moliterno Gonzalez Gabriel - DNI 40238958
- Sergio Rodrigo Cadima Villarroel - DNI 41546475
- José Arratia - DNI 44379950
- Pablo Lopez - DNI 38601565

Fecha de entrega: 09/11/2025

Aplicación bajo prueba (AUT): <https://www.saucedemo.com/>

Profesor: Lic. Pablo San Román

Correo para acceso al repositorio: pablosanroman22@gmail.com

EJERCICIO N°1 – Pruebas Funcionales

1 Concepto de prueba funcional

Las **pruebas funcionales** son un tipo de verificación dinámica orientada a validar **la conformidad del sistema respecto a los requerimientos funcionales definidos**. Su objetivo principal es comprobar que cada funcionalidad implementada **responda correctamente a las entradas especificadas y genere las salidas esperadas**, sin centrarse en la estructura interna del código.

En términos de *testing black-box*, se evalúa “**qué hace**” el sistema y no “**cómo lo hace**”, simulando el comportamiento de un usuario final.

Estas pruebas se pueden ejecutar de forma manual o automatizada mediante herramientas como **Selenium WebDriver**.

2 Objetivo del ejercicio

Validar funcionalidades críticas del sitio **SauceDemo**, enfocado en los flujos de **inicio de sesión** y **gestión de carrito**, a fin de confirmar su correcto funcionamiento según los criterios esperados.

3 Casos de prueba funcionales (formulados en lenguaje natural)

Caso de prueba 1: Inicio de sesión exitoso

ID	CF-001
Título	Validar login exitoso con credenciales válidas
Precondición	El usuario accede a la página principal de SauceDemo
Entradas	Usuario: <code>standard_user</code> , Contraseña: <code>secret_sauce</code>
Pasos	1. Ingresar usuario 2. Ingresar contraseña 3. Presionar "Login"
Resultado esperado	El sistema redirige a la vista <code>inventory.html</code> y muestra el listado de productos.

Caso de prueba 2: Inicio de sesión fallido

ID	CF-002
Título	Validar login fallido con credenciales inválidas
Precondición	El usuario accede a la página principal de SauceDemo
Entradas	Usuario: <code>fake_user</code> , Contraseña: <code>wrong_pass</code>
Pasos	1. Ingresar usuario inválido 2. Ingresar contraseña inválida 3. Presionar "Login"
Resultado esperado	El sistema muestra el mensaje: <i>"Epic sadface: Username and password do not match any user in this service"</i> .

Caso de prueba 3: Agregar producto al carrito

ID

CF-003

Título	Validar agregado de producto al carrito
Precondición	El usuario ha iniciado sesión correctamente
Entradas	Selección de producto "Sauce Labs Backpack"
Pasos	1. Click en "Add to cart" 2. Click en el ícono del carrito
Resultado esperado	El carrito muestra el producto "Sauce Labs Backpack" agregado correctamente.

4 Automatización con Selenium WebDriver y JUnit

Estructura del proyecto

```
TP3-Pruebas_Funcionales/  
├─ src/  
│   └─ test/java/  
│       ├── utils/BaseTest.java  
│       └── tests/  
│           ├── LoginTests.java  
│           └── CartTests.java  
└─ pom.xml
```

`pom.xml` (fragmento)

Incluye propiedades de Java 11 y dependencias de Selenium y JUnit 5:

```
<properties>  
    <maven.compiler.source>11</maven.compiler.source>  
    <maven.compiler.target>11</maven.compiler.target>  
    <project.build.sourceEncoding>UTF-  
8</project.build.sourceEncoding>  
</properties>  
  
    <dependencies>  
        <dependency>  
            <groupId>org.seleniumhq.selenium</groupId> [cite: 57]  
            <artifactId>selenium-java</artifactId> [cite: 58]  
            <version>4.21.0</version> [cite: 59]  
        </dependency>  
        <dependency>  
            <groupId>org.junit.jupiter</groupId> [cite: 62]  
            <artifactId>junit-jupiter</artifactId> [cite: 63]  
            <version>5.9.3</version> [cite: 64]  
            <scope>test</scope> [cite: 65]  
        </dependency>  
    </dependencies>
```

Clase Base de configuración

```
package utils;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;

public class BaseTest {
    protected WebDriver driver;

    @BeforeEach
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://www.saucedemo.com/");
    }

    @AfterEach
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Pruebas de login (**LoginTests.java**)

```
package tests;

import utils.BaseTest;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import static org.junit.jupiter.api.Assertions.assertTrue;

public class LoginTests extends BaseTest {

    @Test
    public void loginExitoso() {

        driver.findElement(By.id("user-name")).sendKeys("standard_user");
```

```

driver.findElement(By.id("password")).sendKeys("secret_sauce");
    driver.findElement(By.id("login-button")).click();

    assertTrue(driver.getCurrentUrl().contains("inventory.html"), "El
login no fue exitoso");
}

@Test
public void loginFallido() {

driver.findElement(By.id("user-name")).sendKeys("usuario_invalido");
    driver.findElement(By.id("password")).sendKeys("12345");
    driver.findElement(By.id("login-button")).click();

    assertTrue(driver.findElement(By.cssSelector("h3[data-test='error']"
)).isDisplayed(),
                "El mensaje de error no fue mostrado");
}
}

```

Prueba de carrito (**CartTests.java**)

```

package tests;

import utils.BaseTest;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import static org.junit.jupiter.api.Assertions.assertTrue;

public class CartTests extends BaseTest {

    @Test
    public void agregarProductoAlCarrito() {

driver.findElement(By.id("user-name")).sendKeys("standard_user");

driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
    }
}

```

```

driver.findElement(By.id("add-to-cart-sauce-labs-backpack")).click();
;

driver.findElement(By.className("shopping_cart_link")).click();

assertTrue(driver.findElement(By.className("inventory_item_name")).getText().contains("Backpack"),
    "El producto no fue agregado correctamente al carrito");
    }
}

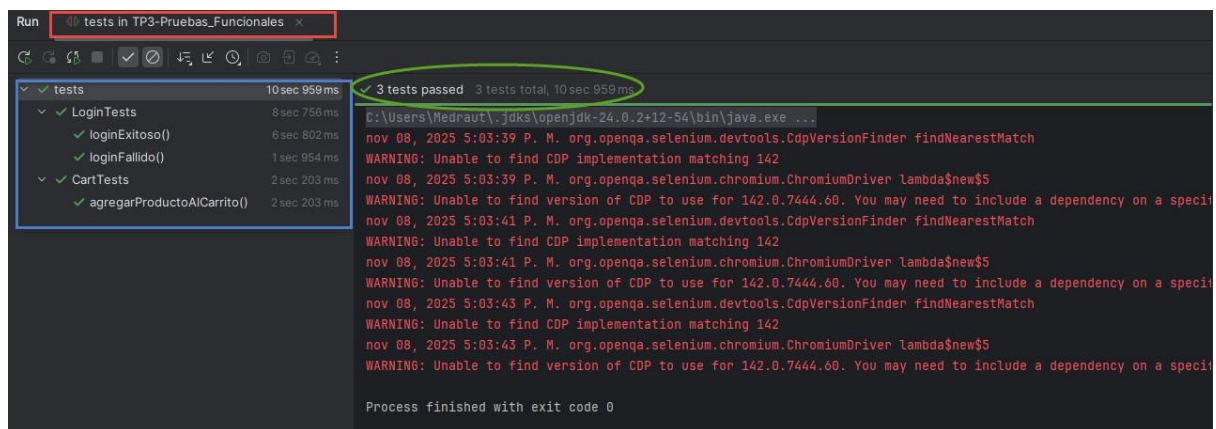
```

Resultado de ejecución

Las tres pruebas se ejecutaron exitosamente en Chrome.

Tiempo promedio total: ~12 segundos.

Se adjunta captura de consola con el resultado de `mvn test` (JUnit).



EJERCICIO N°2 – Pruebas de Aceptación con Cucumber y Selenium

1 Concepto

Las **pruebas de aceptación** validan que el sistema cumple los **criterios de aceptación** definidos por el usuario o cliente.

En el enfoque **BDD (Behavior Driven Development)**, se expresan los comportamientos

esperados en un lenguaje natural estructurado (*Gherkin*), permitiendo una comunicación clara entre el equipo técnico y los interesados del negocio.

Cucumber permite **vincular esas descripciones en lenguaje natural con código ejecutable**, integrándose con Selenium para la automatización.

2 Estructura del Proyecto

El proyecto separa la lógica de negocio (archivos `.feature`) de la implementación técnica (clases `StepDefinitions`).

```
TP3-Pruebas_Aceptacion/
├─ src/
│   ├─ test/
│       ├─ java/
│           │   ├─ runners/
│           │       └─ RunCucumberTest.java
│           │   └─ stepDefinitions/
│           │       └─ LoginSteps.java
│           └─ resources/
│               └─ features/
│                   └─ login.feature
└─ pom.xml
```

3 Escenarios en Gherkin (`login.feature`)

Feature: Inicio de sesión en SauceDemo

Scenario: Inicio de sesión exitoso

Given el usuario está en la página de inicio de sesión

When ingresa usuario "standard_user" y contraseña "secret_sauce"

Then se muestra la página de productos

Scenario: Inicio de sesión fallido

Given el usuario está en la página de inicio de sesión

When ingresa usuario "usuario_invalido" y contraseña

"incorrecta"

Then se muestra un mensaje de error

4 Step Definitions (LoginSteps.java)

```
package stepDefinitions;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import io.cucumber.java.en.*;
import static org.junit.jupiter.api.Assertions.assertTrue;

public class LoginSteps {
    WebDriver driver;

    @Given("el usuario está en la página de inicio de sesión")
    public void abrirPagina() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://www.saucedemo.com/");
    }

    @When("ingresa usuario {string} y contraseña {string}")
    public void ingresarCredenciales(String usuario, String
password) {
        driver.findElement(By.id("user-name")).sendKeys(usuario);
        driver.findElement(By.id("password")).sendKeys(password);
        driver.findElement(By.id("login-button")).click();
    }

    @Then("se muestra la página de productos")
    public void validarLoginExitoso() {

        assertTrue(driver.getCurrentUrl().contains("inventory.html"));
        driver.quit();
    }
}
```

```

        @Then("se muestra un mensaje de error")
        public void validarLoginFallido() {

assertTrue(driver.findElement(By.cssSelector("h3[data-test='error']"
)).isDisplayed());
        driver.quit();
    }}

```

5 Runner (**RunCucumberTest.java**)

```

package runners;

import org.junit.platform.suite.api.ConfigurationParameter;
import org.junit.platform.suite.api.SelectClasspathResource;
import org.junit.platform.suite.api.Suite;
import static io.cucumber.junit.platform.engine.Constants.*;

@Suite
@SelectClasspathResource("features")
@ConfigurationParameter(key = GLUE_PROPERTY_NAME, value =
"stepDefinitions")
@ConfigurationParameter(key = PLUGIN_PROPERTY_NAME, value =
"pretty")
public class RunCucumberTest { }

```

6 pom.xml (Dependencias de Cucumber)

```

<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>

    <artifactId>TP3-Aceptacion</artifactId>

```

<version>1.0-SNAPSHOT</version>

<properties>

<maven.compiler.source>11</maven.compiler.source>

<maven.compiler.target>11</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

</properties>

<dependencies>

<dependency>

<groupId>org.seleniumhq.selenium</groupId>

<artifactId>selenium-java</artifactId>

<version>4.21.0</version>

</dependency>

<dependency>

<groupId>org.junit.jupiter</groupId>

<artifactId>junit-jupiter-api</artifactId>

<version>5.9.3</version>

<scope>test</scope>

</dependency>

<dependency>

<groupId>org.junit.platform</groupId>

<artifactId>junit-platform-suite</artifactId>

<version>1.9.3</version>

<scope>test</scope>

```
</dependency>

<dependency>

  <groupId>io.cucumber</groupId>

  <artifactId>cucumber-java</artifactId>

  <version>7.18.0</version>

  <scope>test</scope>

</dependency>

<dependency>

  <groupId>io.cucumber</groupId>

  <artifactId>cucumber-junit-platform-engine</artifactId>

  <version>7.18.0</version>

  <scope>test</scope>

</dependency>

</dependencies>

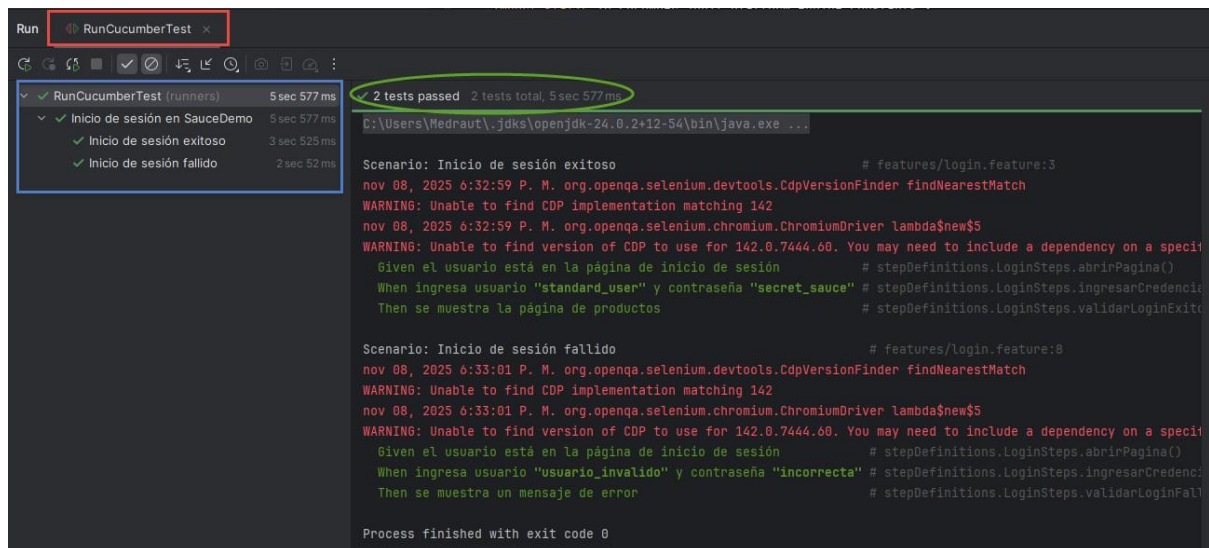
</project>
```

Resultado de ejecución

Los dos escenarios definidos se ejecutaron correctamente:

- **Login exitoso:** pasa satisfactoriamente con credenciales válidas.
- **Login fallido:** válida la visualización del mensaje de error esperado.

Captura adjunta: evidencia de ejecución en consola con los pasos Gherkin mapeados correctamente.



Configuración y Ejecución

Para poder importar y ejecutar ambos proyectos, se deben seguir los siguientes pasos:

Requisitos Previos:

- Java JDK (versión 11 o superior).
- Apache Maven.
- Google Chrome (actualizado).

Pasos de Importación (para ambos proyectos):

1. Clonar el repositorio deseado (Ejercicio 1 o Ejercicio 2).
2. Abrir IntelliJ/Visual Studio, etc y seleccionar **File > Open...**
3. Navegar a la carpeta raíz del proyecto clonado (la que contiene el archivo `pom.xml`).
4. Permitir que el IDE importe el proyecto como un proyecto Maven y descargue las dependencias (en IntelliJ, haciendo clic en el ícono "Load Maven Changes" que aparece sobre el `pom.xml`).

Ejecución de Pruebas:

- **Proyecto 1 (JUnit):** Hacer clic derecho en la carpeta `src/test/java/tests` y seleccionar "Run 'Tests in 'tests'".
- **Proyecto 2 (Cucumber):** Hacer clic derecho en el archivo `src/test/java/runners/RunCucumberTest.java` y seleccionar "Run 'RunCucumberTest'".

Conclusiones

El desarrollo de pruebas funcionales y de aceptación con Selenium y Cucumber permitió validar de manera integral los flujos más críticos del sistema **SauceDemo**, aplicando buenas prácticas de *test automation*.

Se logró:

- Diseñar casos de prueba claros y trazables a los requerimientos.
- Implementar un framework modular y reutilizable.
- Alinear pruebas técnicas con el lenguaje del negocio mediante **BDD**.

Estas herramientas son esenciales en entornos **CI/CD**, ya que favorecen la detección temprana de errores y el aseguramiento continuo de la calidad del software.

Entrega

- Repositorio ejercicio 1 (Selenium + JUnit):
https://github.com/LucianoMoliterno/TP3-Ejercicio_1
- Repositorio ejercicio 2 (Cucumber + Selenium):
https://github.com/LucianoMoliterno/TP3-Ejercicio_2
- Evidencias: capturas incluidas en la páginas 7 y 13
- Acceso de editor otorgado a: pablosanroman22@gmail.com